

MICROCONTROLLERS

Labo 6

Naslag

Gegenereerd op 29/07/2026 uit tdmts.github.io/Microcontrollers

Inhoud

De seriële verbinding

[Het seriële kanaal](#)

Gegevens over de lijn

[Tekens en getallen](#)

[Boodschappen lezen](#)

[Werken met een String](#)

Downloads

Serieel communiceren (C#-project) los document

NASLAG

Het seriële kanaal

Serieel betekent dat de bits van je boodschap één voor één over één draad gaan, in plaats van allemaal tegelijk over acht draden. Dat is traag, maar je hebt er bijna geen draden voor nodig, en daarom zit het op zowat elk apparaat.

Je gebruikt dit kanaal al sinds labo 0. Telkens je `Serial.println()` schreef om te [debuggen](#), ging die tekst serieel over de USB-kabel naar je pc. Aan de andere kant van dat kanaal kan evengoed een tweede Arduino hangen in plaats van een pc.

Drie draden volstaan

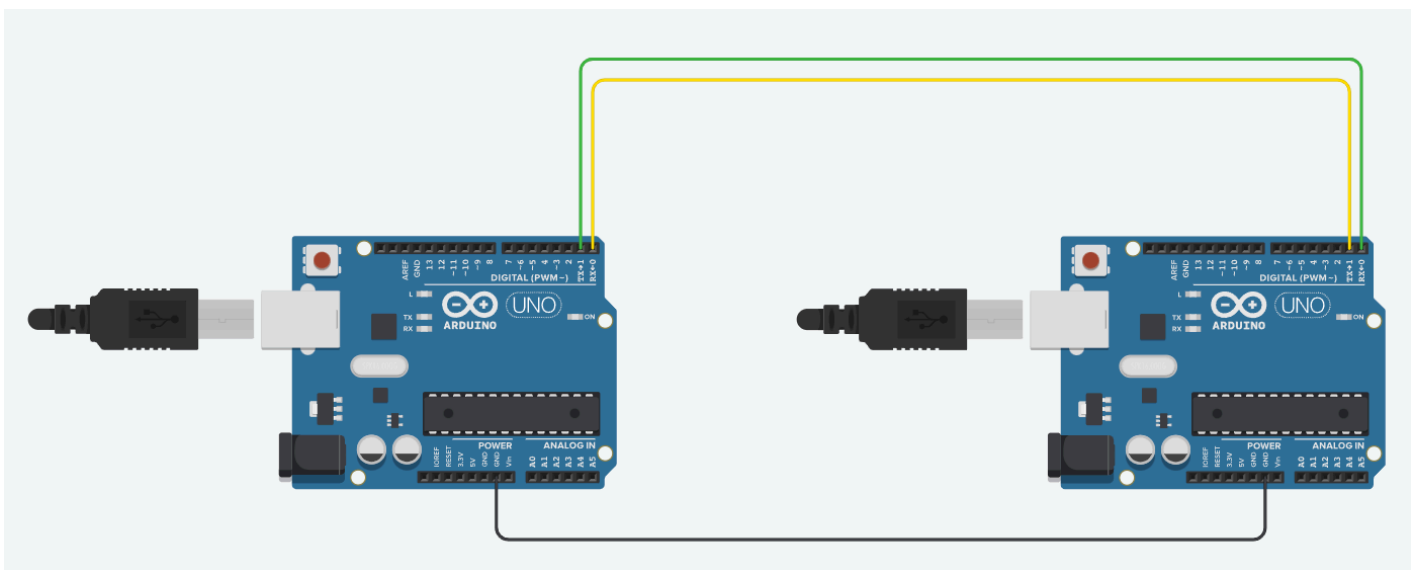
Om twee Arduino's met elkaar te laten praten heb je drie verbindingen nodig.

Draad	Waarvoor
Tx (transmit)	Hierlangs <i>zendt</i> de Arduino. Alles wat je met <code>Serial.print()</code> schrijft, komt hier buiten.
Rx (receive)	Hierlangs <i>ontvangt</i> de Arduino. Alles wat hier binnenkomt, kan je met <code>Serial.read()</code> ophalen.
GND	De gemeenschappelijke massa. Zonder die draad heeft "hoog" bij de ene Arduino geen betekenis voor de andere.

De drie draden van een seriële verbinding

Tx en Rx moeten gekruist

Wat de ene zendt, moet de andere ontvangen. De Tx van Arduino 1 gaat dus naar de Rx van Arduino 2, en de Tx van Arduino 2 naar de Rx van Arduino 1. Verbind je Tx met Tx, dan hangen twee zenders aan elkaar en blijft elke ontvanger onaangesloten.



De groene draad gaat van Tx naar Rx, de gele van Rx naar Tx. De zwarte draad onderaan is de gemeenschappelijke massa.

Rx en Tx zijn pin 0 en pin 1

Op de Arduino UNO zijn Rx en Tx de eerste twee digitale pinnen. Kijk maar naar het opschrift op de print, daar staat **RX←0** en **TX→1**.

Daarom heb je die twee pinnen in alle vorige labo's nooit gebruikt voor een led of een knop. Ze zijn bezet.

OP ECHTE HARDWARE DEELT JE USB-KABEL PIN 0 EN 1

De USB-poort waarlangs je een programma uploadt, hangt op precies dezelfde Rx en Tx. Op een echt bord mislukt het uploaden dus zolang er draden op pin 0 en 1 zitten. Trek die twee draden er uit, upload je programma, en steek ze daarna terug.

In TinkerCAD heb je daar geen last van, want daar is er geen echte USB-kabel. Onthoud het toch voor de dag dat je dit op een echt bord bouwt.

Beide kanten moeten even snel praten

De baudrate is het aantal bits per seconde dat over de lijn gaat. Bij `Serial.begin(9600)` zijn dat er 9600. Er loopt geen kloksignaal mee over de draad, dus de ontvanger gaat ervan uit dat de bits aan het afgesproken tempo binnenkomen.

Staan de twee kanten niet op dezelfde baudrate, dan meet de ontvanger op de verkeerde momenten en krijg je onleesbare tekens, net zoals wanneer je seriële monitor op een andere snelheid staat dan je `Serial.begin()`.

```
1 void setup()
2 {
3     Serial.begin(9600); // exact dezelfde waarde in beide programma's
4 }
```

CPP

ACHTER SERIAL ZIT EEN UART

Het onderdeel dat dit werk doet, heet een **UART**: universal asynchronous receiver-transmitter. Het zit in je ATmega328P ingebouwd, naast het geheugen en de timers. Jij geeft er met `Serial.print()` een byte aan, en de UART schuift die bits daarna zelf een voor een naar buiten op het afgesproken tempo, terwijl je `loop()` gewoon verder draait. Aan de overkant doet een tweede UART het omgekeerde: die vangt de bits op, zet ze terug samen tot een byte en houdt ze bij tot jij `Serial.read()` doet. De *a* van asynchroon is wat hierboven staat: er loopt geen klok mee over de draad, dus telt elke UART zijn eigen tijd af en moeten beide kanten op dezelfde baudrate ingesteld staan.

Zowat elke microcontroller heeft er minstens één aan boord, en veel grotere chips hebben er drie of vier. De modules die je erop aansluit gebruiken dezelfde manier van praten: gps-ontvangers, bluetoothmodules, vingerafdruklezers, 4G-modems en heel wat sensoren hebben geen andere aansluiting dan een TX, een RX en een massa. Wat je hier tussen twee Arduino's legt, is dus ook de manier waarop je later zo'n module aan je bord hangt.

Ook in afgewerkte apparaten blijft die aansluiting zitten. Open je een router, een decoder of een industriële sturing, dan vind je op de print vaak vier contacten op een rij, soms met een header erin en meestal zonder. Daar hangt de fabrikant een kabel aan om te volgen wat het toestel doet terwijl het opstart, en om erbij te kunnen wanneer het scherm of het netwerk niets meer geeft. Zo'n verbinding heeft genoeg aan één draad per richting en een gemeenschappelijke massa, en werkt zonder driver en zonder netwerk.

In de video hieronder wordt zo'n aansluiting op een echt toestel opgezocht en aangesloten, tot de opstartboodschappen binnenlopen. De video is Engelstalig.

Video.

<https://www.youtube.com/watch?v=ZmZuKA-Rst0>

EÉN KANAAL, TWEE LUISTERAARS

De seriële monitor hangt aan dezelfde Tx-lijn. Alles wat je naar de andere Arduino stuurt, zie je dus ook in je eigen monitor verschijnen. Dat is handig, want zo zie je wat je verstuurt.

Tekens en getallen

Over een seriële lijn gaan bytes, en die bytes stellen *tekens* voor. Dat verklaart waarom de andere kant 49 leest terwijl jij 1 verstuurde.

Een getal in het geheugen

Schrijf je dit:

```
byte waarde = 100;
```

CPP

dan staat er in het geheugen één byte, die er binair uit ziet als **01100100**. Dat is honderd, in bits.

Hetzelfde getal op de lijn

Verstuur je dat getal nu:

```
Serial.print(100);
```

CPP

dan komen er aan de andere kant **drie** bytes binnen in plaats van de ene byte **01100100**, namelijk de tekens **'1'**, **'0'** en **'0'**. **Serial.print()** zet je getal eerst om naar tekst, en verstuurt die tekst teken per teken.

Welke byte bij welk teken hoort, ligt vast in de **ASCII-tabel**. Dit zijn de rijen die je in dit labo nodig hebt:

Teken	Waarde	Opmerking
'0'	48	De cijfers staan op een rij, dus '7' is 48 + 7 = 55.
'1'	49	
'9'	57	
'A'	65	De hoofdletters staan ook op een rij, tot 'Z' = 90.
','	58	Het scheidingsteken dat je in dit labo gebruikt.
'\n'	13	Carriage return, zie hieronder.
'\n'	10	Line feed, zie hieronder.

De ASCII-waarden die je hier nodig hebt

Hieronder staat de volledige tabel met alle 128 tekens. De kolom **Dec** is de waarde die je met `Serial.read()` binnenkrijgt, de kolom **Chr** is het teken dat erbij hoort.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com

De volledige ASCII-tabel. Bron: LookupTables.com

HET TEKEN '1' HEEFT DE WAARDE 49

Dit is het klassieke struikelblok. Stuur je `Serial.print(1)` en lees je aan de andere kant met `Serial.read()`, dan krijg je 49 te zien, niet 1. Je hebt het teken ontvangen, niet de waarde. Om er weer een getal van te maken moet je het omrekenen, en daar dient [het String-type](#) voor.

WAAROM UITGEREKEND 49

Toen ASCII in 1963 werd vastgelegd, kregen de tien cijfers hun plaats niet toevallig. Ze staan zo dat je het cijfer zelf terugvindt in de onderste helft van de byte. Schrijf 49 binair uit en je krijgt `00110001`: de vier bits achteraan zijn `0001`, en dat is 1.

Dat klopt voor alle tien de cijfers. '0' is 48 of `00110000`, en '9' is 57 of `00111001`. De bovenste vier bits zijn bij elk cijfer dezelfde, de onderste vier zijn de waarde. Apparatuur uit die tijd moest daardoor niets omrekenen om een cijfer af te drukken, en jij hebt er ook geen tabel voor nodig: '1' - '0' is 49 - 48, en dat is 1.

print of println

`Serial.println()` doet exact hetzelfde als `Serial.print()`, maar plakt er achteraf twee extra tekens aan: `'\r'` (13) en `'\n'` (10). Samen betekenen die twee "nieuwe lijn", precies wat er gebeurt als je op Enter duwt.

Instructie	Bytes op de lijn	Hoeveel
<code>Serial.print(100);</code>	49 48 48	3
<code>Serial.println(100);</code>	49 48 48 13 10	5

Wat er echt over de lijn gaat

Die twee extra tekens zijn het enige waaraan de ontvanger kan zien *waar je boodschap ophoudt*. Stuur je met `print()` drie getallen na elkaar, dan komt er aan de andere kant `121317` binnen en valt daar 12, 13 en 17 niet meer uit af te leiden.

DE REGEL VAN DIT LABO

De zender gebruikt **altijd** `Serial.println()`, dus één boodschap per lijn. De ontvanger leest **altijd** een volledige lijn tegelijk. Hou je je daaraan, dan werkt alles in dit labo.

Hoe je zo'n lijn uitleest, staat op [Boodschappen lezen](#).

Boodschappen lezen

Tot nu toe ging je seriële verkeer maar één richting uit, van de Arduino naar jouw scherm. Hier draait dat om. Je leert hoe je binnenkomende gegevens opvangt, en waarom je ze altijd per lijn leest.

De ontvangstbuffer

Binnenkomende bytes worden automatisch opgeslagen in een stuk geheugen dat de **buffer** heet, 64 bytes groot. Jouw programma haalt ze daar op wanneer het uitkomt.

Dat betekent twee dingen. Je moet *niet* exact op het juiste moment staan te kijken, want de bytes blijven in de buffer staan. Maar je mag ook niet te lang wegblijven, want als er meer dan 64 bytes binnenkomen voor je iets ophaalt, valt het oudste eruit en ben je het kwijt.

Is er al iets binnen?

`Serial.available()` geeft je het aantal bytes dat op dit moment in de buffer klaarstaat. Is dat 0, dan is er niets binnengekomen en heeft lezen geen zin.

```
1 void loop()
2 {
3     if (Serial.available() > 0)
4     {
5         // pas hier is er echt iets te lezen
6     }
7 }
```

CPP

Eén byte tegelijk met read()

`Serial.read()` haalt precies één byte uit de buffer en verwijdert die daar. Je krijgt de *waarde* van dat teken terug, dus een getal tussen 0 en 255. Is de buffer leeg, dan krijg je -1.

```

1  void loop()
2  {
3      if (Serial.available() > 0)
4      {
5          int teken = Serial.read();
6          Serial.println(teken); // stuurde de andere kant "100", dan zie je 49, dan 48, dan 48
7      }
8  }

```

Dat is nuttig om te begrijpen wat er echt over de lijn gaat, maar je wil zelden zo werken. Je moet dan zelf de tekens opvangen, bijhouden en weer tot een getal samenstellen. Zie [Tekens en getallen](#) voor waarom 49 daar staat.

Een lijn tegelijk met readStringUntil()

Veel handiger is dat de Arduino zelf tekens verzamelt tot hij het einde van de lijn tegenkomt. Dat doet `Serial.readStringUntil('\n')`. Je krijgt alles wat er vóór de `'\n'` stond terug als `String`.

```

1  void loop()
2  {
3      if (Serial.available() > 0)
4      {
5          String regel = Serial.readStringUntil('\n');
6          regel.trim(); // haalt de \r en eventuele spaties weg
7
8          Serial.println(regel);
9      }
10 }

```

De `trim()` is nodig. De zender schrijft `println()` en zet er dus `'\r'` én `'\n'` achter. `readStringUntil()` stopt bij de `'\n'`, dus die `'\r'` zit nog aan je tekst geplakt. Vergeet je de `trim()`, dan is `regel` gelijk aan `"LED:1\r"` in plaats van `"LED:1"`, en dan klopt je vergelijking niet meer.

⚠ GEEF READSTRINGUNTIL() ÉÉN TEKEN

`readStringUntil()` wil één teken, tussen enkele aanhalingstekens. Schrijf je `readStringUntil('\r\n')` met twee tekens ertussen, dan geeft de compiler een waarschuwing en doet je programma iets wat je niet bedoelde. Neem de `'\n'` en ruim de rest op met `trim()`.

Hoelang wacht readStringUntil?

Als de `'\n'` nooit komt, blijft `readStringUntil()` wachten, standaard één seconde. Zolang staat je `loop()` stil. Zolang de zender `println()` gebruikt, merk je daar niets van, want de `'\n'` komt altijd. Wil je die wachttijd korter, dan zet je in je `setup()`:

```
Serial.setTimeout(100); // wacht hoogstens 100 ms op het einde van een lijn
```

CPP



SAMENGEVAT

- `Serial.available()` om te kijken *of* er iets is.
- `Serial.readStringUntil('\n')` plus `trim()` om een volledige boodschap op te halen.
- `Serial.read()` alleen als je echt teken per teken wil kijken.

Werken met een String

Een **String** is een variabele met tekst erin, net zoals een **int** er een geheel getal in heeft. Je hebt er in dit labo een nodig, want een boodschap die binnenkomt is tekst voor je er een getal van maakt.

Je maakt er een aan zoals elke andere variabele. Let op de hoofdletter S, want dat is de naam van het type.

```
String commando = Serial.readStringUntil('\n');
```

CPP

Wat je ermee kan

Een String kan meer dan een gewone variabele, want je kan hem van alles vragen. Dat doe je met een punt achter de naam, net zoals bij **Serial.print()**.

Dit zijn de functies die je in dit labo nodig hebt. Hieronder staan ze allemaal in één programma dat je kan uploaden. Typ in de seriële monitor een boodschap met een dubbelepunt erin, bijvoorbeeld **LED:1**, en je ziet van elke instructie het resultaat verschijnen. Achter elke regel staat in commentaar wat je krijgt als je precies **LED:1** typt.

```
1 void setup()
2 {
3     Serial.begin(9600);
4     Serial.println("Typ een boodschap zoals LED:1 en druk op Enter.");
5 }
6
7 void loop()
8 {
9     if (Serial.available() > 0)
10    {
11        String regel = Serial.readStringUntil('\n'); // wat je typte, met de \r er nog aan
12
13        Serial.print("Lengte voor trim: ");
14        Serial.println(regel.length()); // 6, want de \r telt mee
15
16        regel.trim(); // geeft zelf niets terug, verandert regel ter plaatse
17
18        Serial.print("Lengte na trim: ");
19        Serial.println(regel.length()); // 5
20
21        int scheiding = regel.indexOf(':');
22
23        Serial.print("Dubbelepunt op plaats: ");
24        Serial.println(scheiding); // 3, tellen begint bij 0
25
26        Serial.print("Komma op plaats: ");
27        Serial.println(regel.indexOf(',')); // -1, want er staat geen komma in
28
29        String sleutel = regel.substring(0, scheiding); // tot net voor plaats 3
30        String waarde = regel.substring(scheiding + 1); // vanaf plaats 4 tot het einde
31
32        Serial.print("Sleutel: ");
33        Serial.println(sleutel); // LED
34
35        Serial.print("Waarde: ");
36        Serial.println(waarde); // 1
37
38        Serial.print("Waarde als getal: ");
39        Serial.println(waarde.toInt()); // 1
40
41        // Omzetten naar een getal lukt niet altijd zoals je hoopt
42        String meting = "20.12";
43
44        Serial.print("toFloat op 20.12: ");
45        Serial.println(meting.toFloat()); // 20.12
46
47        Serial.print("toInt op 20.12: ");
48        Serial.println(meting.toInt()); // 20, alles vanaf de punt valt weg
49
50        String onzin = "twintig";
51
52        Serial.print("toInt op twintig: ");
```

```
53     Serial.println(onzin.toInt());    // 0, en geen enkele foutmelding
54 }
55 }
```

Die drie laatste zijn de moeite om te onthouden. `toInt()` en `toFloat()` geven je nooit een foutmelding. Lukt het omzetten niet, dan krijg je 0 terug. Je kan dus niet zien of er echt een 0 binnenkwam of iets wat helemaal geen getal was.

⚠ TOFLOAT() VERWACHT EEN DECIMALE PUNT

`toFloat()` verwacht een decimale **punt**, zoals in `20.12`. Stuur je `20,12` door, dan stopt hij bij de komma en houd je 20 over. Schrijf je zender dus altijd met een punt, ook al zeg je in het Nederlands "twintig komma twaalf".

Twee Strings vergelijken

Met `==` vergelijk je de inhoud, precies zoals je zou verwachten. Hoofdletters tellen mee, dus `"LED"` is niet hetzelfde als `"led"`.

```
1  if (sleutel == "LED")
2  {
3      // ...
4  }
```

CPP

Er bestaat ook `sleutel.equals("LED")`. Dat doet exact hetzelfde, want de `==` van een String roept intern `equals()` op. Neem gerust `==`, dat leest het vlotst. Wil je hoofdletters negeren, dan is er `equalsIgnoreCase()`.

i ALS JE JAVA KENT, VERGEET DIE REGEL HIER

In Java moet je `equals()` gebruiken, want daar vergelijkt `==` of het twee keer dezelfde plaats in het geheugen is en niet of er hetzelfde in staat. In Arduino heeft de String-klasse een eigen `==` die de inhoud vergelijkt. Hier zijn de twee dus echt inwisselbaar.

Waar het wel misgaat is bij tekst die geen String is. Vergelijk je twee `char*` met `==`, dan vergelijk je adressen en krijg je bijna altijd `false`. Zolang minstens één van de twee kanten een String is, zoals hier, zit je goed.

SLEUTEL:WAARDE uit elkaar halen

In dit labo ziet elke boodschap er zo uit: eerst een sleutel die zegt *waarover* het gaat, dan een dubbelepunt, dan de waarde. Dus **TEMP:20.12**, **HUMIDITY:40** of **MODE:AUTO**.

De ontvanger zoekt de dubbelepunt, knipt de lijn daar in twee, kijkt met een **if** welke sleutel het is, en zet pas daarna de waarde om naar het juiste type. Dat laatste kan je pas als je de sleutel kent, want bij **TEMP** hoort een kommagetal, bij **HUMIDITY** een geheel getal, en bij **MODE** helemaal geen getal.

CPP

```
1 void loop()
2 {
3     if (Serial.available() > 0)
4     {
5         String regel = Serial.readStringUntil('\n');
6         regel.trim();
7
8         int scheiding = regel.indexOf(':');
9
10        if (scheiding > 0) // zonder dubbelepunt is het geen geldige boodschap
11        {
12            String sleutel = regel.substring(0, scheiding);
13            String waarde = regel.substring(scheiding + 1);
14
15            if (sleutel == "TEMP")
16            {
17                float temperatuur = waarde.toFloat();
18                Serial.print("Het is ");
19                Serial.print(temperatuur);
20                Serial.println(" graden.");
21            }
22            else if (sleutel == "HUMIDITY")
23            {
24                int vochtigheid = waarde.toInt();
25                Serial.print("Vochtigheid: ");
26                Serial.print(vochtigheid);
27                Serial.println("%");
28            }
29            else if (sleutel == "MODE")
30            {
31                Serial.print("Modus staat op ");
32                Serial.println(waarde); // hier blijft het gewoon tekst
33            }
34        }
35    }
36 }
```

Die **if (scheiding > 0)** staat er omdat **indexOf()** je -1 geeft als er geen dubbelepunt in staat. Zou je dan toch verder doen, dan rekent **substring(0, -1)** met onzin. Er komt vroeg of laat rommel binnen op een

seriële lijn, dus vang dat op.

WAAROM EEN SLEUTEL DE MOEITE WAARD IS

Stuur je kale getallen, dan moet de ontvanger tellen om te weten waar hij zit: het eerste getal is de temperatuur, het tweede de vochtigheid, en zo verder. Mist hij er één, dan klopt alles wat daarna komt niet meer, en hij merkt er niets van.

Met een sleutel voor elke waarde kan dat niet gebeuren. Elke boodschap zegt zelf wat ze is, dus één gemiste lijn heeft geen gevolgen voor de volgende.

String en het geheugen van je UNO

Een String past zijn lengte aan terwijl je programma draait. Hij vraagt daarvoor telkens een stuk geheugen, en geeft het weer vrij als hij niet meer nodig is. Een UNO heeft daar 2 kB voor. Vraag en geef je lang genoeg stukken van wisselende grootte, dan raakt dat geheugen versnipperd, waardoor er nog wel genoeg vrij is maar nergens een aaneengesloten stuk dat groot genoeg is. Je programma loopt dan vast zonder foutmelding.

Dat klinkt erger dan het voor jou is. Het patroon dat je in dit labo gebruikt, is net het veilige:

```
1 String regel = Serial.readStringUntil('\n'); // aangemaakt
2 regel.trim();
3 // ... hier gebruik je hem ...
4                                     // en aan het einde van de loop weer opgeruimd
```

CPP

Er leeft hier altijd maar één String tegelijk, en die wordt aan het einde van elke ronde weer vrijgegeven. Het stuk dat vrijkomt is precies het laatst gevraagde, dus het sluit weer aan bij de rest. Zo blijft er niets versnipperd achter.

Dit is riskant	Doe liever dit
Meerdere Strings tegelijk in leven houden, met verschillende levensduur	Eén String per ronde, en hem meteen weer laten verdwijnen
Een String beetje bij beetje laten groeien met += in een lus	<code>regel.reserve(32);</code> in je <code>setup()</code> , zodat hij meteen groot genoeg is
Lange zinnen aan elkaar plakken met + om ze daarna te tonen	Meerdere keren <code>Serial.print()</code> of <code>lcd.print()</code>

Wat het geheugen wel versnipperd

💡 WIL JE ZIEN HOEVEEL ER NOG VRIJ IS?

Met deze functie meet je hoeveel geheugen er nog tussen je variabelen en je stack zit. Print het elke paar seconden. Blijft het getal stabiel, dan zit je goed. Zakt het traag maar zeker, dan lekt er iets.

```
1  int vrijGeheugen()  
2  {  
3      extern int __heap_start, *__brkval;  
4      int hier;  
5  
6      return (int) &hier - (__brkval == 0 ? (int) &__heap_start : (int) __brkval);  
7  }
```

CPP

Bouw je ooit iets dat weken aan een stuk moet draaien, dan laat je **String** beter achterwege en lees je je tekens in een vaste **char**-array. Die heeft altijd dezelfde grootte, dus er valt niets te versnipperen. Het kost meer regels code, en daarom blijft dat hier achterwege. Voor de oefeningen van dit labo, en voor zowat alles wat je in dit vak bouwt, is **String** de juiste keuze.